

4.1.1. Pandas - series creation and manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases



seriesMa...

Submit

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5
6 s=pd.Series(numbers)
7 grouped = s.groupby(s % 2 == 0).mean()
8
9 # Display the mean of even and odd numbers with labels
10 grouped.index = ['Even' if is_even else 'Odd' for is_even in
    grouped.index]
```

Average time
0.007 s
6.83 ms

Maximum time
0.016 s
16.00 ms

3 out of 3 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case 1 16 ms Debug

Expected output
1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:
Odd 5.0
Even 6.0
dtype: float64

Actual output
1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:
Odd 5.0
Even 6.0
dtype: float64

Terminal Test cases

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.

datafram... Submit

```
1 import pandas as pd
2
3 # Provided dictionary of lists
4 data = {
5     'Name': ['Alice', 'Bob', 'Charlie'],
6     'Age': [25, 30, 35],
7 }
8
9 # Convert the dictionary to a DataFrame
10 df = pd.DataFrame(data)
11
```

Average time 0.049 s 49.00 ms Maximum time 0.056 s 56.00 ms

1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed

Test case 1 56 ms Debug

Expected output	Actual output
Original DataFrame:	Original DataFrame:
.....Name..Age	Name..Age
0...Alice...25	0...Alice...25
1...Bob...30	1...Bob...30
2...Charlie...35	2...Charlie...35
New name: Susan	New name: Susan

Terminal Test cases

4.1.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:
Refer to the displayed test cases for better understanding.

Sample Test Cases

```
1 import pandas as pd
2
3 # Read the text file into a DataFrame
4 file = input()
5 data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age",
6 "Grade"])
7
8 print("First five rows:")
9 print(data.head())
10 av=round(data['Age'].mean(),2)
11 print("Average age:",av)
```

Average time

0.022 s

22.00 ms

Maximum time

0.034 s

34.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 34 ms

Debug

Expected output

studentdata.txt

First five rows:

--Name--Age--Grade--

0--John--25--A--

1--Alin--22--B--

2--Emma--24--A--

Actual output

studentdata.txt

First five rows:

--Name--Age--Grade--

0--John--25--A--

1--Alin--22--B--

2--Emma--24--A--

Terminal

Test cases

1.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 df['Date'] = pd.to_datetime(df["Date"])
10 df["Month"] = df["Date"].dt.strftime("%Y-%m")
11 df["Total Sales"] = df["Quantity"] * df["Price"]
```

Average time
0.016 s
16.33 ms

Maximum time
0.034 s
34.00 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 34 ms Debug

Expected output	Actual output
sales_data.csv	sales_data.csv
Best-month: 2025-01	Best-month: 2025-01
Total-sales: \$1210.00	Total-sales: \$1210.00

Terminal

Test cases

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases



monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9
10 product_sales = df.groupby("Product")["Quantity"].sum()
11 # Find the product with the highest total quantity sold
```

Average time

0.013 s

13.00 ms

Maximum time

0.026 s

26.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 26 ms

Debug

Expected output

sales_data.csv

Best-selling product: Product A

Total quantity sold: 23

Actual output

sales_data.csv

Best-selling product: Product A

Total quantity sold: 23

Terminal

Test cases

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Explorer frequently...

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 date_products = {}
```

Average time 0.013 s 13.00 ms Maximum time 0.026 s 26.00 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 26 ms Debug

Expected output	Actual output
sales_data.csv	sales_data.csv
Product A and Product B: 2 times	Product A and Product B: 2 times
Product A and Product C: 2 times	Product A and Product C: 2 times

Terminal Test cases

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

Submit

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 city_sales = df.groupby("City")["Quantity"].sum()
10 best_city=city_sales.idxmax()
11 # Display the result
```

Average time
0.012 s
12.33 ms

Maximum time
0.025 s
25.00 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 25 ms Debug

Expected output
sales_data.csv
City sold the most products: Los Angeles

Actual output
sales_data.csv
City sold the most products: Los Angeles

Terminal Test cases

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

Explorer

titanicDat...

1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # 1. Display the first 5 rows of the dataset
8 print(data.head())
9
10 # 2. Display the last 5 rows of the dataset
11 print(data.tail())

Average time
0.113 s
113.00 ms

Maximum time
0.113 s
113.00 ms

1 out of 1 shown test case(s) passed

Test case 1 113 ms Debug

Expected output
-- PassengerId -- Survived -- Pclass -- ... -- Fare -- Cabin -- Embarked
0 -- 1 -- 0 -- 3 -- ... -- 7.2500 -- NaN -- S
1 -- 2 -- 1 -- 1 -- ... -- 71.2833 -- C85 -- C
2 -- 3 -- 1 -- 3 -- ... -- 7.9250 -- NaN -- S

Actual output
-- PassengerId -- Survived -- Pclass -- ... -- Fare -- Cabin -- Embarked
0 -- 1 -- 0 -- 3 -- ... -- 7.2500 -- NaN -- S
1 -- 2 -- 1 -- 1 -- ... -- 71.2833 -- C85 -- C
2 -- 3 -- 1 -- 3 -- ... -- 7.9250 -- NaN -- S

Terminal Test cases

4.2.6. Titanic Dataset Analysis and Data Cleaning - 201:00

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

Pas sen ger id	Sur vive d	Pcla ss	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Fare	Cab in	Em bark ed

Sample Data:

titanicDat... Submit

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7
8 # 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
9 data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)
10
11 # 2. Convert 'Sex' to numeric (male: 0, female: 1)
```

Average time0.045 s45.00 ms

Maximum time0.045 s45.00 ms

1 out of 1 shown test case(s) passed

Test case 145 ms Debug

Expected output

Actual output

29.69911764705882

29.69911764705882

14.4542

14.4542

3...491

3...491

1...216

1...216

2...184

2...184

Name: Pclass, dtype: int64

Name: Pclass, dtype: int64

Terminal Test cases

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

Pas sen ger id	Sur vive d	Pcla ss	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Far e	Cab in	Em bark ed

Sample Data:

titanicDat...

Submit

1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7 data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
8 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
9
10 print(data.groupby('Pclass')['Survived'].mean())
11

Average time
0.053 s
53.00 ms

Maximum time
0.053 s
53.00 ms

1 out of 1 shown test case(s) passed

Test case 1 53 ms Debug

Expected output
Pclass
1 --- 0.629630
2 --- 0.472826
3 --- 0.242363
Name: Survived, dtype: float64
Embarked_S
0 0.500000

Actual output
Pclass
1 --- 0.629630
2 --- 0.472826
3 --- 0.242363
Name: Survived, dtype: float64
Embarked_S
0 0.500000

Terminal

Test cases